🎓 QA101 — n8n Quickstart

n8n Academy · Completed · Updated 2026-11-09

Overview

Course notes for **n8n Quickstart** from n8n Academy: workflow basics, Data Tables, filtering, Code node, scheduling, export/import, data processing, and building an **AI Agent** with memory and tools.

Course Map

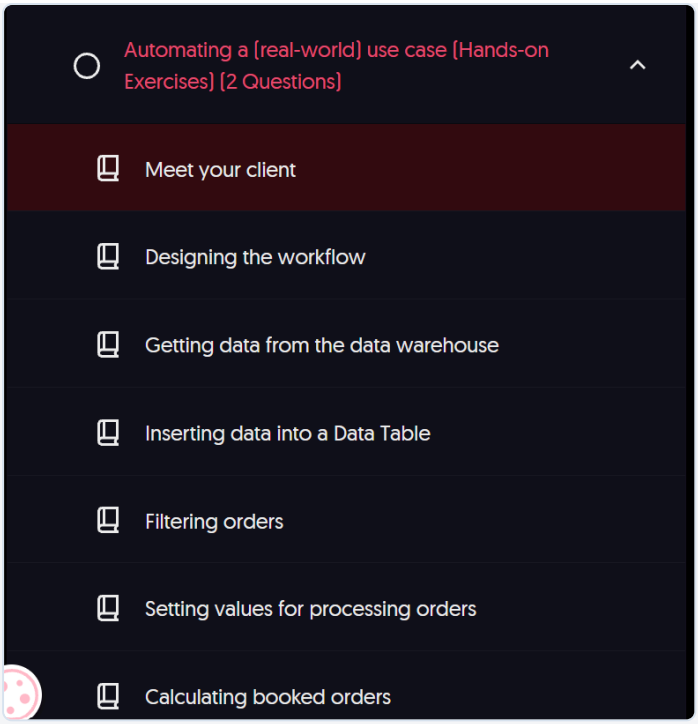
Section	Focus	Units
PART 1	Getting Started — workflow, Data Table, filter, Code, Discord, schedule, publish, export/import	1–10
PART 2	Working with Data — data structures, HTML/XML, dates, binary, split, merge, loop, debugging, error handling, exercises	1–12
PART 3	Building an AI Agent — memory, tools, system prompt, testing	1–11
Quiz Bank	QA101 quizzes & exam (16 questions)	—

🗺️ Table of Contents

1. **PART 1 — Getting Started**: workflow design, Data Table, filter, Edit Fields, Code node, Discord, Schedule Trigger, publish, export/import.
2. **PART 2 — Working with Data**: data structures, HTML/XML, date & time, binary data, split, merge, looping, debugging, error handling, and exercises.
3. **PART 3 — Building an AI Agent**: the memory problem, Simple Memory, tools, system prompt, and testing the agent.
4. **Quiz Bank & Wrap-up**: 16 quiz questions with answers, next-step programs, and a node cheat sheet.

Opening

It started with curiosity — let's dive in.



This way, we truly understand how the whole flow works.

PART 1 — Getting Started

1. Meeting Part 1 — Who Is Nathan?

👤 Who Is Nathan?

An employee facing a weekly task that is exhausting, repetitive, and mind-numbing.

⚠️ What Is the Main Problem?

Every week, Nathan has to gather sales data from the company's *data warehouse* — a *legacy*, in-house system that **has no CSV export feature**.

📋 What Are His Routine Tasks?

1. Count the total number of orders with the `Booked` status so it can be announced on the company Discord every Monday.
2. Compile a list of orders with the `Processing` status so the Sales Managers can review them.

🚨 Challenges & Risks

Because the work is done manually and demands high attention to detail, it is highly error-prone. Nathan often loses focus, mistypes numbers, finishes late, and has even been reprimanded by his manager.

---

2. Designing the Workflow — Part 2

🎯 Main Goal

Build an **8-node** automation to process weekly sales data from the internal *legacy data warehouse* that has no CSV export feature.

🛠️ The 6-Step Workflow

#	Step	Purpose
1	**Get Data**	Retrieve sales data (*order ID*, *status*, *value*, *employee name*) from the *data warehouse* via API.
2	**Filter Data**	Separate orders by status (`Processing` vs `Booked`).
3	**Calculate Sum**	Calculate the total value of all orders with `Booked` status.
4	**Discord Notification**	Send the `Booked` order report to the company Discord channel.
5	**Data Table Storage**	Insert `Processing` order details into n8n Data Tables for the Sales Managers' *follow-up*.
6	**Schedule Automation**	Schedule the *workflow* to run automatically every Monday morning.

🌐 Core Integration Concept: HTTP Request Node

> **Why use the HTTP Request node?**
>
> Because the old *data warehouse* has no **Dedicated Node** (a purpose-built integration node such as Discord or Google Sheets), the **HTTP Request Node** is used as a versatile solution to communicate directly with the system's **API Endpoint**.

⚙️ Node Configuration `GetDataFromWarehouse`

Parameter	Setting / Value
Node Type	HTTP Request (connected to *Manual Trigger*)
Method	`GET`
URL	`https://learn.app.n8n.cloud/webhook/courses/n8n-quickstart/company-warehouse`
Send Headers	`true` (*Using Fields Below*)
Header Name	`x-assessment-id`
Header Value	`<YOUR_ASSESSMENT_ID>`
Authentication	`Generic Credential Type`
Generic Auth Type	`Header Auth`
Credential Setup	Name: `X-API-KEY` | Value: *(Practice API Key)*

---

3. Inserting Data into a Data Table — Part 3

💡 Core Concepts

- **n8n Data Table**: a built-in internal database similar to a *spreadsheet* that lives right inside n8n.
- **No Integration Needed**: no external services or API credentials required.
- **Requirement**: the *workflow* and the *data table* must be in the **same workspace**.

⚙️ Node Configuration `UpsertOrders`

Parameter	Value
Node	Data Table (connected from `GetDataFromWarehouse`)
Operation	`Upsert row(s)` — add new data or update existing rows
Data Table	`orders`
Must Match	`Any condition`
Conditions	`orderId` → `equals` → `{{ $json.orderID }}` *(Expression Mode)*
Mapping Mode	`Map Automatically`

🚨 Key Points

- **Auto Mapping**: the *field* names from the HTTP Request output must match the table column names **exactly**.
- **Upsert Behavior**: prevents duplicate data when re-running tests.
- **Built-in Columns**: n8n adds them automatically (`id` , `createdAt` , `updatedAt`).

> **Next Step**
>
> Add a **Filter** node to separate orders with `Processing` status (into the Data Table) from `Booked` status (whose total value is calculated).

---

4. Filtering Orders — Part 4

`#n8n` `#workflow` `#automation` `#learning-notes`

> 💡 **Core Concepts**
>
> - **Filtering Data:** sort incoming data using **conditional logic** (an *if-then-else* rule) before inserting it into a database or Data Table.
> - **If Node:** splits the *workflow* into **2 branches** (`True` & `False`) based on the result of a comparison.
> - **Switch Node:** use this when you need to route data to **more than 2 conditions/branches** (*multiple output connectors*).
> - **Data Efficiency (Best Practice):** only store the *fields/columns* you actually need. Storing unused data wastes compute, slows processing, and consumes storage.

⚙️ Node Configuration `CheckOrderStatus`

- **Node Type:** `If`
- **Connected From:** `HTTP Request`

Parameter	Setting / Value
Value 1	`{{ $json.orderStatus }}` *(Expression Mode)*
Operation	`String` → `is equal to`
Value 2	`processing`

> 🚨 **Key Points & Performance Impact**
>
> - **Output flow:**
> - True Branch → passes `processing` orders to the **Data Table**.
> - False Branch → passes orders with other statuses (e.g. `booked`) so their values can be calculated.
> - **Optimization Note:** with small data (e.g. 14 *records*) the impact is not noticeable, but adding extra columns/data across thousands of *records* will add up and affect performance.

> ⏩ **Next Step**

> Configure the **False** branch to calculate the total accumulated value of **Booked** orders.

---

5. Setting Values for Processing Orders — Part 5

> 💡 **Core Concepts**
>
> - **Edit Fields (Set) Node:** a node that selects, changes, filters, or adds data before passing it to the next node (such as a *Data Table* or *database*).
> - **Data Reduction:** only pass the data that is actually needed (e.g. just `employeeName` and `orderID`) from *processing orders* to keep storage and performance efficient.
> - **Data Manipulation Options:** 1. **Pull Existing Data:** select specific *fields* from earlier nodes. 2. **Completely New Data:** create new *fields/columns* with fixed values or new calculations that did not exist before. 3. **Overwrite Data:** replace existing values.

⚙️ Configuration & Workflow

- **Node Type:** `Edit Fields (Set)`
- **Connected From:** the `True` branch of `CheckOrderStatus` (If Node)
- **Target Fields:** `employeeName` & `orderID`

Feature / Action	Description
Select / Set Fields	Determines which *fields* are allowed to pass to the next node.
Data Clean-up	Removes non-essential columns from the `HTTP Request` output.
Preparation	Prepares a clean data structure before the *insert/upsert* into the Data Table.

> 🚨 **Key Points & Best Practice**
>
> - **Data Expectations:** nodes such as *spreadsheets* or *databases* require a precise data structure. Use the Edit Fields node to tidy it up first.
> - **Performance & Storage:** dropping irrelevant *fields* reduces the data *payload*, making encryption/transfer faster.

> ⏭️ **Next Step (Next Unit)**
>
> Use **Custom JavaScript Code** inside a dedicated node to calculate and sum the total accumulated value of **Booked** orders (the *False* branch).

---

6. Calculating Booked Orders (Code Node) — Part 6

> 💡 **Core Concepts & Code Node Modes**
>
> - **Code Node:** a flexible node that lets you run custom logic using **JavaScript** or **Python**.

- **2 Code Node Modes:** 1. **Run Once for All Items:** processes all input items at once as a group. Ideal for totals, *sums*, or *aggregation*. 2. **Run Once for Each Item:** runs the code separately, one item at a time.
- **Native n8n Data Structure:** data flowing between nodes is an **Array of Objects** with the required nested shape: `[{ json: { ... }, binary: { ... } }]`.

⚙️ Configuration

- **Node Type:** `Code` (choose *Code in JavaScript*)
- **Connected From:** the `False` branch of `CheckOrderStatus` (If Node)
- **Execution Mode:** `Run Once for All Items`

Example implementation

```
// Mode: Run Once for All Items
// Adjust the field name to match your HTTP Request output
let total = 0;

for (const item of $input.all()) {
  total += Number(item.json.orderValue);
}

return [{ json: { totalBooked: total } }];
```

The result is a **single item** containing the total value of `Booked` orders, ready to send to Discord.

---

7. Notify the Team (Discord) — Part 7

`#n8n` `#workflow` `#automation` `#discord` `#webhook` `#expressions` `#learning-notes`

💡 Core Concepts

- **Notification Automation:** automatically send processed results (a summary report of *booked* orders) to the team's communication platform (Discord), saving time and avoiding tedious manual work.
- **Webhook Integration:** use the **Discord Node** with a *Webhook* connection to send messages without complex bot setup.
- **Dynamic Expressions (`{{ ... }}`):** build the message content automatically by combining plain text with variable data from previous nodes.
- **Cross-Node Referral:** pull a specific parameter from a different node on the *canvas* (e.g. taking the *Assessment ID* from the header of the `GetDataFromWarehouse` node).

⚙️ Discord Node Configuration

Parameter	Value
Node Type	`Discord`
Connection Type	`Webhook`
Credential Status	`Create New Credential`
Credential Name	`n8n Quickstart S1 Discord Webhook account`
Webhook URL	`https://learn.app.n8n.cloud/webhook/courses/n8n-quickstart/notify-team`
Operation	`Send a Message`

---

8. Scheduling the Workflow — Part 8

`#n8n` `#workflow` `#automation` `#scheduling` `#schedule-trigger` `#learning-notes`

> 💡 **Core Concepts**
>
> - **Scheduling Automation (`Schedule Trigger`):** lets a *workflow* run automatically in the background on a specific date, time, or interval without a manual trigger.
> - **Multi-Trigger Support:** one *workflow* can have more than one *trigger*. Keep the *Manual Trigger* for on-demand testing, and use the *Schedule Trigger* for recurring automation.
> - **Timezone Sensitivity:** execution accuracy depends heavily on the timezone configured in *Workflow Settings* or on the n8n instance.

⚙️ Node Configuration `TriggerMondays9am`

- **Node Type:** `Schedule Trigger`
- **Node Name:** `TriggerMondays9am`
- **Connected To:** the HTTP Request node `GetOrdersFromWarehouse`

Parameter	Value	Notes
Trigger Interval	`Weeks`	Weekly execution interval.
Weeks Between Triggers	`1`	Runs once every week.
Trigger on weekdays	`Monday`	Runs every Monday (remove the default *Sunday*).
Trigger at Hour	`9am`	09:00 AM.
Trigger at Minute	`0`	Minute 0.

🕒 Timezone & Automatic Execution Requirements

1. **Timezone Hierarchy:** - The *Schedule Trigger* first checks and uses the timezone in **Workflow Settings**. - If the *workflow* timezone is not set, the system *falls back* to the **n8n Instance** timezone.

2. **Workflow Status Requirement (*Publish*):** - The *Schedule Trigger* will **NOT** run automatically if the *workflow* is still a draft (*unpublished*). - The *workflow* must be **published / activated (*Publish*)** first for the scheduled trigger to work in the background.

> 🚨 **Important Notes**
>
> - **Naming Best Practice:** give nodes specific names (like `TriggerMondays9am`) to keep the flow readable as the *workflow* grows.
> - **Next Step:** once the *workflow* is ready and fully integrated, the next stage is to **Publish Workflow** and learn how to read the **Execution Log**.

---

9. Publishing & Examining the Workflow — Part 9

`#n8n` `#workflow` `#automation` `#executions` `#settings` `#debugging` `#learning-notes`

> 💡 **Core Concepts**
>
> - **Workflow Executions:** represent a single run of the *workflow* from the first node to the last.
> - **Execution Log & Debugging:** historical records used to monitor success/failure, inspect actual data at each node, and troubleshoot issues.
> - **Workflow Settings Customization:** per-*workflow* settings that can overwrite the global defaults of the n8n instance (such as *timezone*, log storage, and *error handling*).
> - **Data Wrangling Elimination:** turn complex manual work (*data wrangling*) into a consistent 8-node automation running in the background.

📊 Executions Window Columns

Column	Description & Purpose
Name	The name of the executed *workflow*.
Started At	The *timestamp* when execution started.
Status	Execution status (`Waiting` , `Running` , `Succeeded` , `Cancelled` , `Failed`) and duration.
Execution ID	A unique identifier for each execution session.

> **Filtering Options**
>
> Execution history can be filtered by status (`Any Status` , `Success` , `Failed` , `Cancelled` , `Running` , `Waiting`) or by a specific *workflow*.

⚙️ Key Features in Workflow Settings

Setting	Purpose & Configuration
Execution Logic	*Multi-branch* execution logic (recommended default: `v1`).

Setting	Purpose & Configuration
Error Workflow	Routes to another *workflow* that sends a dedicated notification if the main *workflow* fails.
Sub-workflow Call	Controls whether this *workflow* can be called by the `Execute Sub-workflow` node.
Timezone	The *workflow's* specific timezone (critical for `Schedule Trigger` accuracy).
Save Executions	Options to save production execution logs (`Failed` / `Successful`) and `Manual` executions.
Save Execution Progress	Enables *resume workflow* from the point of failure (can slow down processing).
Redact Execution Data	Masks/hides sensitive data from production/manual log views.
Estimated Time Saved	Estimates time saved per execution to calculate automation ROI (*Return on Investment*).

🗺️ n8n UI Navigation

```
Editor Canvas ⟶ Executions Tab (Current Workflow Only)
Overview      ⟶ Executions Tab (Global, All Workflows)
Editor UI (...) ⟶ Settings (Access Workflow Settings)
```

---

10. Exporting & Importing Workflows — Part 10

`#n8n` `#workflow` `#automation` `#export` `#import` `#json` `#learning-notes`

> 💡 **Core Concepts**
>
> - **JSON File Format:** n8n *workflows* are stored and transferred as local JSON data structures.
> - **Flexible Data Sharing:** makes *backup*, collaboration between developers, and reuse of external *workflow* templates easy.
> - **Two Main Methods:** export/import can be done through the **Editor UI Menu** (file/URL) or directly on the **Canvas** (keyboard copy/paste shortcuts).

⚙️ n8n Export & Import Methods

Access Type	Export Method (Save/Share)	Import Method (Load/Add)
Via Editor UI Menu *(Top-Right Three Dots)*	Choose **Download** → saves the *workflow* as a `.json` file on your computer.	Choose **Import from File** *(JSON file)* or **Import from URL** *(public link)*.
Via Editor Canvas *(Keyboard Shortcuts)*	Select all nodes → press `Ctrl+C` (Windows) / `CMD+C` (Mac) to copy the JSON code.	Paste the JSON code directly onto an empty *canvas* area using `Ctrl+V` / `CMD+V` .

> 🚨 **Important Notes**
>
> - **Credential Security:** when exporting a *workflow* to JSON, n8n automatically **does NOT** include sensitive credential data (such as *API keys* or passwords). Anyone importing that *workflow* must configure their own credentials.

- **Canvas Shortcut:** `Ctrl+C` / `Ctrl+V` on the *canvas* is the fastest way to move some or all nodes between different *workflow tabs*.

PART 2 — Working with Data

1. Working with Data & Data Structures in n8n

`#n8n` `#javascript` `#data-structure` `#array` `#json`

💡 Core Concepts & Data Structures

- **n8n's Main Role (ETL):** works like an *Extract, Transform, Load* tool where data flows between nodes in a structured format that every node can read and interpret.
- **Standard Data Format:** all data sent between nodes in n8n is an **array of JSON objects**, where each element is called an **item**.
- **Arrays:** ordered lists of values starting at index 0.
- Example: `["Leonardo", "Michelangelo", "Donatello", "Raphael"]` — `Donatello` is at index `2`.
- **Objects:** data structures that store *key-value pairs*.

Object example

```
{
  "name": "Alice",
  "email": {
    "personal": "alice@home.com",
    "work": "alice@company.com"
  }
}
```

---

2. Creating and Referencing Data with the Code Node

`#n8n` `#javascript` `#data-structure` `#array` `#json` `#code-node` `#data-referencing`

💡 Creating Data Manually in the Code Node

- **Standard Format & Auto-Wrap:** every object in an n8n array is wrapped with a `json` key. In modern n8n versions, this `json` wrapper is added automatically if you return a plain array of objects.
- **Nested Objects:** used to group more detailed data or sub-properties. Created by wrapping key-value pairs in an additional set of curly braces `{}` (e.g. separating personal and work email).
- **Data Creation Example:**
- Create a contact array variable (Alice and Bob) containing `name` and `email` properties (`personal` & `work`).
- *Return* that variable so it can be processed by the next node.

Data creation example

```
const contacts = [
  {
    name: 'Alice',
    email: {
      personal: 'alice@home.com',
      work: 'alice@company.com',
    },
  },
  {
    name: 'Bob',
    email: {
      personal: 'bob@home.com',
      work: 'bob@company.com',
    },
  },
];

return contacts;
```

n8n automatically wraps each object into `{ json: { ... } }`.

⚙️ Referencing Data Across Nodes (Code Node & Upstream Nodes)

- `$input.all()`: retrieves all incoming items from the immediately preceding node for manipulation (e.g. adding a new column `items[0].json.workEmail = ...`).
- `$('NodeName')`: retrieves data from a node further upstream.

```
// Add a new field from the previous node
const items = $input.all();
items[0].json.workEmail = items[0].json.email.work;

return items;
```

🔍 Item Selection Options on `$('NodeName')`

Option	Purpose	Example
`.first()`	Get only the first item. Used when the node outputs a single item or you only need one result.	`$('NodeName').first().json.field`
`.last()`	Get the very last item.	`$('NodeName').last().json.field`
`.all()`	Get all items as an array.	`$('NodeName').all()`

Option	Purpose	Example
`.item`	Get the item at the same position/index as the item currently being processed. Used when two nodes have the same number of items.	`$('NodeName').item.json.field`

📌 Node Naming Best Practices

- **Descriptive & Self-Documenting:** change default names (like `HTTP Request 2`) to names that describe their purpose (like `GetCountries` or `CalculateTotal`).
- **Use CamelCase / PascalCase:** removing spaces from node names makes expressions called via `$('NodeName')` easier and cleaner to write.
- **Sticky Notes:** use sticky notes on the canvas to explain the overall workflow.

---

3. HTML & XML Data Processing in n8n

`#n8n` `#html-node` `#xml-node` `#web-scraping` `#data-transformation`

💡 Using the HTML Node (Web Scraping)

- **Main Purpose:** extract content or structured information from web pages (*web scraping*) using *CSS Selectors*.
- **Key Configuration:**
- **Operation:** *Extract HTML Content*.
- **Source Data & Property:** points to the HTML source data (e.g. from an *HTTP Request* node) in the appropriate JSON property.
- **Extraction Values:** defines the column name (*Key*) and the target HTML element path using a *CSS Selector* (e.g. `.post .item-title a` to grab article titles).
- **Return Array Toggle:** enable it to capture all matching elements/occurrences on the page, or disable it if you only need the first result.

⚙️ Using the XML Node (Format Conversion)

- **Main Purpose:** convert data back and forth between **XML and JSON**.
- **Use Case:** very useful when you need to bridge or send data between *web services* or *legacy systems* that require XML, while n8n data is in JSON format.

---

4. Date, Time & Interval Data in n8n

`#n8n` `#date-time` `#luxon` `#wait-node` `#workflow-automation`

> 💡 **Date & Time Data Types**
>
> - **Supported formats:** `DATE`, `TIME`, `DATETIME`, `TIMESTAMP` (Unix / ms), and `YEAR` in various formats.
> - **Schedule Trigger:** schedules workflow execution at a specific time, interval, or duration.

- **Wait Node:** delays workflow execution (based on a specific time, interval, or webhook call) to avoid incomplete data from slow services.

⚙️ Date Processing Nodes & Methods

1. **Date & Time Node (No-Code / Visual Approach):** - **Purpose:** change formats, calculate/add dates, round, and compare dates. - **Rounding Rules:**
   - *Round Up:* forces the date forward to the start of the next month regardless of the mid-month boundary (e.g. `April 10` jumps straight to `May 1`).
   - *Round Down / Floor:* drops the remaining days and moves back to the start of the current month (e.g. `April 28` becomes `April 1`).
   - *Round Nearest:* rounds to the nearest month based on the mid-month midpoint.
   - **Important Option (`Include Input Fields`):** keeps the old data columns (`id`, `name`, `email`, etc.) from being removed and displays them alongside the new result columns.
2. **Luxon Expressions (Code / Flexible Approach):** - Uses the Luxon JavaScript library bundled with n8n. - Applied directly in expressions (such as in the Edit Fields node) using `.toDateTime().plus()` or `.toDateTime().minus()`.

📌 Workflow Exercise Summary (Date Operations)

Node	Purpose
GetCustomerData	Retrieves customer data including the creation date (`created`).
RoundDate *(Date & Time Node)*	Rounds the `created` date up (*Round Up*) to the start of the next month and produces the `newDate` column.
CheckAfter1960 *(If Node)*	Splits the data flow based on whether `newDate` is after `1960-01-01`.
Add5days *(Date & Time Node — True Branch)*	Adds 5 days to the `newDate` date using the visual node.
Add20Years *(Edit Fields Node — False Branch)*	Adds 20 years to the `newDate` date using the Luxon expression: `{{ $json.newDate.toDateTime().plus(20, 'years') }}`.

---

5. Binary Data in n8n

`#n8n` `#binary-data` `#http-request` `#edit-image` `#file-handling`

💡 Understanding & Handling Binary Data

- **What is Binary Data?** Files such as images, PDFs, audio, and video that are represented in binary form and require conversion before they can be read/processed.
- **File Path Notes:**
- Cloud / Docker installation: use `/tmp/filename` as the file path.
- npm installation: use `~/filename`.

⚙️ Core Nodes for Binary Data

Node	Purpose
HTTP Request	Transfers files from web *resources* and APIs.
Read/Write Files from Disk	Reads and writes files directly (n8n *self-hosted* only).
Convert to File	Converts input data (e.g. JSON) into files such as CSV or XLSX.
Extract From File	Converts binary formats into JSON (e.g. extracting text from a PDF).
Edit Image	Performs image manipulation such as rotate, *crop*, *resize*, and *compositing*.

📌 Exercise Summary (Downloading & Editing an Image)

- **GetDogURL (HTTP Request 1):** fetches a random dog image URL from an API (`https://learn.app.n8n.cloud/webhook/apis/dog-images`).
- **GetDogImage (HTTP Request 2):** downloads the binary image file using the URL expression from the previous node (`{{ $json.message }}`).
- **Rotate180 (Edit Image Node):** rotates the image by 180 degrees (flipped vertically/horizontally) using the *Rotate* operation.

---

6. Splitting Data in n8n

`#n8n` `#split-out` `#data-transformation` `#api-handling`

> 💡 **Why Split Data?**
>
> - **Problem:** when fetching data from an API, the data often arrives as a single item containing a *nested array* (e.g. one object with a `results` field holding 20 records). n8n reads this as just 1 item.
> - **Solution:** *downstream nodes* usually need to process each record separately, so the array must be split.

⚙️ How the Split Out Node Works

- **Main Purpose:** takes a *field* containing an *array* and extracts each element into its own standalone item. This is one of the most frequently used data transformation nodes in n8n.
- **Key Settings:**
- **Field To Split Out:** the name of the *field* containing the *array* to split (e.g. `results`).
- The node reads that *field*, creates one output item per array element, and optionally keeps other fields from the original item alongside the new items.

📌 Exercise Summary (Split Pokemon)

- **Initial Condition:** Pokémon data from the API arrives as a single item containing an *array*.
- **SplitPokemon (Split Out Node):** configured with `Field To Split Out` set to `results`.
- **Execution Result:** becomes several separate items (e.g. 5 separate items), each carrying its own `name` and `url` fields to be processed independently by the next node.

7. Merging Data in n8n

`#n8n` `#merge-node` `#data-transformation` `#sync`

> 💡 **Why Merge Data?**
>
> - **Main Purpose:** combine data from different sources into a single data set.
> - **Common Use Cases:** combining customer data from a CRM with order data from a database, or syncing data between systems (removing duplicates / updating records).

🔄 Data Synchronization Types

1. **One-way Sync:** data flows from one main source to one or more destinations (changes in the source are pushed to the destination, but not the reverse).
2. **Two-way Sync:** data flows in both directions between systems (changes in either system are synchronized to the other).

⚙️ Merge Node Join Methods

- **Append:** combines items sequentially (all items from Input 1 followed by all items from Input 2).
- **Combine:** merges data with several sub-options:
- *Merge by Fields:* matches items from both inputs based on specific *field* values (similar to a database *JOIN*).
- *Merge by Position:* pairs items from both inputs based on their position/order (*index*).
- *Combine All:* creates every possible combination of items from both inputs (*Cartesian product*).
- **Choose Branch:** selects the output from one specific input and ignores the other.

> ⚠️ **Important Note When Filling Parameters**
>
> If you need to reference *nested values* in the *Input 1 Field* and *Input 2 Field* parameters of the *Merge node*, write the *property key* using **dot-notation** as plain text (e.g. `name.common`), **not** as an expression (don't use `{{ $json.name.common }}`).

8. Looping & Batch Processing

`#n8n` `#looping` `#batch-processing` `#data-transformation` `#automation`

💡 Core Data Processing Concept in n8n

- By default, a node in n8n receives data items (similar to rows in a table), processes them, and outputs the results.
- **Automatic Iteration:** most n8n nodes run automatically for each item.
- *Example:* if a Slack node is connected to a database node returning 5 customer records, the Slack node automatically runs 5 times to send 5 messages without needing a manual *loop*.

⚙️ Executing Nodes Once

- If you don't want a node to process *all* incoming items, enable the **Execute Once** option on that node's *Settings* tab.
- The node will only process the **first item** and ignore the rest.

🔁 Creating Manual Loops & Node Exceptions

1. When Do You Need a Manual Loop?

n8n iterates automatically for most nodes, but there are some **exceptions** where a node does not automatically iterate over all items:

- **Code Node** in *Run Once for All Items* mode.
- **HTTP Request:** when handling API *pagination*, a manual loop is needed to fetch page by page.
- **RSS Read:** only runs once for the requested URL.
- **Execute Workflow Node** in *Run Once for All Items* mode.
- **Some Database Nodes** (CrateDB, MS SQL, MongoDB, QuestDB, TimescaleDB): only run *insert* and *update* operations once.

2. How to Create a Manual Loop (Classic Method)

- Connect the *output* of a node back to the *input* of a previous node.
- Use an **If** node to check when the loop should stop (*stopping condition*).

📦 Splitting Data into Batches

Used to process large data volumes gradually or to avoid API rate limits.

Using the Loop Over Items Node

- **Purpose:** splits incoming data into several *batches* of configurable size, then processes each *batch* through the connected nodes.
- **Per-Item Processing:** set **Batch Size = 1** to process items one by one.
- **Advantage:** the loop stops automatically once all data has been divided and processed. **No If node is needed** as a *stopping condition*.

---

9. Checking & Debugging Failed Workflows

`#n8n` `#debugging` `#executions-log` `#troubleshooting` `#error-handling`

📜 Executions Log

- **Purpose:** n8n's main tool for viewing workflow run history and tracking errors.
- **Access:** through the left panel of the Editor UI or the *Executions* tab when a workflow is open.
- **Information Displayed:**
- Workflow name
- Execution start time

- Status (*Succeeded, Failed, Running, Waiting,* or *Cancelled*)
- Execution duration
- Execution ID

⚙️ Saving Successful Executions

- **Default:** n8n *only* saves **failed** executions.
- **How to Enable Success Logs:** open *Workflow Settings* → set **Save Successful Production Executions** to **Yes**.
- **Usefulness:** helps verify that a workflow runs correctly or inspect output data from past executions.

🔍 Inspecting & Fixing Failed Executions

1. Inspecting Failed Executions

- Selecting a failed execution opens a *read-only* mode.
- The failing node is marked in **red**.
- Click the red node to view the error message details and the data being processed when the failure occurred.
- Use the **Editor | Executions** toggle at the top to switch quickly between edit mode and log review.

2. Debugging in Editor (Pinning Data)

- Click **Debug in editor** on the failed execution to copy that execution's data into the workflow editor.
- **Pinned Data:** the data that caused the *error* is automatically *pinned* in each node.
- **Benefit:** you can rerun individual nodes or the whole workflow using the exact data that caused the *error* without triggering an execution from scratch.
- **Final Step:** fix the issue, test against the *pinned data*, then *unpin* the data once the problem is resolved.

---

10. Catching & Handling Errors

`#n8n` `#error-handling` `#retry-on-fail` `#stop-and-error` `#error-trigger` `#automation`

> 💡 **Error Handling Levels in n8n**
>
> n8n provides 3 levels of *error* handling, from the most specific (per node) to the broadest (whole workflow): 1. **Node-level Settings:** error handling settings on each node in the *Settings* tab. 2. **Stop and Error Node:** a dedicated node to intentionally stop the workflow when data is invalid. 3. **Error Workflows:** a separate workflow that runs automatically when a *production workflow* fails, to send notifications.

⚙️ 1. Node-Level Error Handling

A. "On Error" Setting

Controls workflow behavior when a specific node fails:

Option	What Happens	When to Use It
Stop Workflow *(Default)*	The entire workflow stops immediately and the execution is marked *failed*.	When the node is critical and subsequent nodes must not run if it fails.
Continue	The workflow continues to the next node. The failed node outputs *empty data*.	When the node is optional and subsequent nodes can still work without that data.
Continue Using Error Output	The node adds a new red *output connector* (*error output*). Successful data goes through the normal output, failed data through the red output.	When you want to handle *errors* directly inside the workflow (e.g. send a notification or take an alternative path).

B. Retry on Fail

- **Purpose:** automatically retry node execution on *transient* failures (*timeouts*, API *rate limits*).
- **Configuration:** found in the node's *Settings* tab. You can set the number of retries (up to 5 times) and the wait interval in milliseconds (ms).

🛑 2. Stop and Error Node

- **Purpose:** intentionally stop the workflow and throw an *error* when a specific condition is met.
- **Error Types:**
- **Error Message:** a custom text message explaining the error.
- **Error Object:** a structured data object with more detailed error information.
- **Use Case:** place it after a validation check (e.g. an **If** node) to catch corrupted/invalid data from a third-party API before it enters the next process.

🚨 3. Error Trigger & Error Workflows

A separate background workflow that handles failure notifications for *production workflows*.

Setup & Configuration Steps:

1. Create a new workflow and add an **Error Trigger** node as the first node.
2. Connect it to a notification node (e.g. Discord, Slack, Gmail) to alert the team.
3. Save the **Error Workflow**.
4. Open the main workflow you want to monitor → go to **Workflow Settings** → select the **Error Workflow** you created in the *Error Workflow* option.

> **Important Notes**
>
> - A workflow with an *Error Trigger* node uses itself as its own error handler by default.
> - **Cannot be tested manually:** it only activates during *production* (non-manual) executions.
> - **Reusability:** one *Error Workflow* can be shared by many main workflows.

---

11. Hands-on Exercise: Workflow 1 — Merging Customer Data

`#n8n` `#hands-on` `#data-enrichment` `#merge-node` `#data-tables`

> 🎯 **Main Goal**
>
> Automatically fill (*enrich*) the empty `region` and `subregion` columns in the `customers` Data Table by combining internal data (database) with external data (n8n Academy API).

🛠️ Workflow Outline

1. **Get Customers Data (Local Database)** - Retrieve 10 customer records from the `customers` Data Table. - At this stage, the `region` and `subregion` columns are still empty.
2. **Get Countries Data (External API)** - Retrieve the list of country names along with `region` and `subregion` information from the n8n Academy API (`https://learn.app.n8n.cloud/webhook/countries`).
3. **Merge Data (Matching Data)** - Combine both datasets using the **Merge** node (Mode: `Combine` → `Matching Fields`). - Match the `customerCountry` column (from customer data) with the `name` column (from the country API). - **Result:** customer data is now enriched with the matching regional information.
4. **Update Customers Table (Write Back)** - Update the `customers` Data Table with the completed regional data. - Use the **Conditions** marker (`id` Equals `{{ $json.id }}`) so n8n updates the correct row for each customer ID.

> 💡 **Key Takeaways**
>
> - **Data Enrichment:** the technique of completing incomplete local data using references from an external API.
> - **Merge by Fields / Matching Fields:** how to match two different data sources using identical reference values (*keys*) (e.g. Country Name).
> - **Update Conditions:** a mandatory requirement when updating Data Table rows so n8n updates the specific row (*match by ID*) instead of overwriting someone else's data.

---

12. Hands-on Exercise: Workflow 2 (Part 1) — Retrieving Order Data

`#n8n` `#hands-on` `#http-request` `#data-table` `#merge-node` `#data-enrichment`

> 🎯 **Main Goal**
>
> Retrieve 30 transaction records (*order data*) from the *company warehouse* via API, then enrich them by combining customer profile data from the `customers` Data Table (including the `region` and `subregion` fields filled in Workflow 1).

🛠️ Workflow Outline ("Generating Reports")

1. **GetOrderData (HTTP Request Node)** - Retrieve order data from the warehouse API (`https://learn.app.n8n.cloud/webhook/courses/n8n-quickstart/company-data`). - Apply authentication via **Header Auth** (`x-assessment-id`) to secure the API call. - **Output:** 30 order records (containing `orderID` , `customerID` , `orderPrice` , `quantity` , `productCategory` , `orderDate` , etc.).

2. **GetCustomers (Data Table Node — Parallel Branch)** - Runs in parallel from the *Manual Trigger*. - Retrieves all rows from the `customers` Data Table (resource: *Row*, operation: *Get*). - **Output:** 10 customer records complete with `region` and `subregion` columns.
3. **Merge (Merge Node)** - Connects `GetOrderData` (Input 1) and `GetCustomers` (Input 2). - **Settings:** Mode `Combine` → `Matching Fields` (or *Merge by Fields*). - **Matching Key:** pair `customerID` (Input 1) with `customerID` (Input 2). - **Final Output:** 30 transaction records now enriched with customer location information (`country` , `region` , `subregion` , `email`).

> 💡 **Key Takeaways**
>
> - **Header Auth Credential:** sends a unique identification key (`x-assessment-id`) in the HTTP Request *Header* so the server allows data access.
> - **Parallel Branching:** runs the HTTP Request and Data Table nodes simultaneously from one *Trigger* before merging them at the *Merge* node.
> - **Data Relationship (One-to-Many Join):** one customer (`customerID`) can have many transactions. The *Merge* node automatically matches the `customerID` in each transaction with the correct customer profile.

PART 3 — Building an AI Agent

> **Context**
>
> This section continues the **Customer service agent** workflow built in the previous unit. The focus: add **memory** so the agent remembers conversations, then give it **tools** so it can read real data.

1. The Problem: An Agent With No Memory

Your agent can hold a conversation, but try this: tell it your name, wait for a response, then ask "*What's my name?*" It will not be able to answer. It might apologize or guess, but it has no idea what you said 10 seconds ago.

Why? Because the **AI Agent has no memory**. Each message is treated as a completely new conversation. The model receives only your latest message and the *system prompt*, with no record of what came before.

For a customer service agent, this is a problem. A customer might say "*I'm calling about order 5*" and then follow up with "*What's the status?*" Without memory, the agent has already forgotten which order they were asking about.

---

2. Test It Yourself

Open the **Customer service agent** workflow you built in the previous unit.

1. Open the chat window by clicking the **Chat** button at the bottom of the *canvas*.
2. Type `Hi, my name is Nathan` and press **Enter**. Wait for the response.
3. Now type `What's my name?` and press **Enter**.

The agent will not know your name. It might say something like "*I don't have access to that information*" or take a guess. Either way, it clearly did not remember what you told it one message ago.

---

3. Adding Simple Memory

n8n solves this with **memory nodes**. The simplest option is **Simple Memory**, which stores a window of recent messages so the agent can see what has already been said in the current conversation.

1. On the *canvas*, click the **+** button underneath the **Memory** connector on the AI Agent node. A list of memory options will appear.
2. Select **Simple Memory**.
3. The default settings are fine. Simple Memory stores the last **5 interactions** by default — enough for most conversations. You can increase this later if you need the agent to remember more context.
4. Close the node to return to the *canvas*.

---

4. How Memory Works

Simple Memory does not give the model a brain. What it actually does is **append recent messages to the prompt** each time the agent runs. When you send "*What's my name?*", the model receives something like:

- **System prompt:** "*You are a customer service agent...*"
- **User message 1:** "*Hi, my name is Nathan*"
- **Assistant response 1:** "*Hello Nathan, how can I help you today?*"
- **User message 2:** "*What's my name?*"

The model can "remember" your name because it can see the earlier exchange right there in the prompt. The window size (default 5 interactions) controls how far back this context goes. Older messages get dropped to keep the prompt from growing too large.

> **Memory Limitations**
>
> - **Not permanent storage.** If a conversation goes on long enough, earlier messages will fall out of the window.
> - **Lost on restart.** Simple Memory is stored only in memory, so it disappears when the workflow stops or the n8n instance restarts.
> - **For production:** use database-backed memory such as **Postgres Chat Memory** or **Redis Chat Memory** so conversations persist across restarts.

---

5. Save the Workflow

Save the workflow before moving on. In the next unit, you will give the agent **tools** so it can actually look up customer and order data instead of guessing.

---

6. Giving Your Agent Tools

Why Tools Matter

Right now your agent can chat and remember what you said, but if you ask "*What region is customer 10 in?*", it will either make something up or admit it has no idea. The model has no access to your data — it is just generating text based on patterns it learned during training.

Tools change that. A tool is a connection between the AI Agent and an external data source. When the agent receives a question it cannot answer from its own knowledge, it can decide to call a tool, read the result, and use that information in its response. You do not tell it when to use the tool: **the agent decides on its own** based on the question and the tool description you provide.

In this unit, you will give your customer service agent two tools:

1. Read customer data from the Data Table you built in Section 2.
2. Fetch order data from an API endpoint.

---

7. Adding the Customers Data Table Tool

The first tool gives the agent access to customer records: names, countries, emails, regions, and subregions.

1. Open your **Customer service agent** workflow.
2. Click the **+** button underneath the **Tool** connector on the AI Agent node. A list of available tool types will appear.
3. Search for `Data Table` and select **Data Table Tool**.
4. In the tool settings, set the **Operation** to **Get**.
5. For **Data Table**, select the **customers** table you created in Section 2.
6. Rename the node to **GetCustomers**.
7. Close the node to return to the *canvas*.

> **The Importance of Tool Descriptions**
>
> The tool description is how the agent knows when to use that tool. A vague description leads to the agent calling the wrong tool or not calling one at all. Be specific about **what data the tool returns**.

---

8. Updating the System Prompt

Now that the agent has tools, update the *system prompt* to reflect what it can actually do.

1. Open the **AI Agent** node and find the **System message** field you set up earlier.
2. Replace the prompt with something like:

```plaintext
You are a customer service agent. You have access to a customer database and an order database.

When asked about customers, use the GetCustomers tool to look up their information. When asked about orders, prices, employees, or product categories, use the GetOrderData tool.

Be concise and helpful. If the data does not contain what the user is asking for, say so. Do not make up information.
```

3. Close the node.

> **Why mention tools in the prompt?**
>
> Telling the agent which tool to use for which type of question is not strictly required (it can figure it out from the tool descriptions), but it makes the agent **more reliable**. Think of it as giving a new employee clear instructions rather than hoping they work it out themselves.

---

9. Testing the Agent with Tools

Open the chat window and try asking questions that require the agent to use its tools.

Example questions:

- `What region is customer 10 in?`

- `How many orders does customer 3 have?`
- `Who is the employee assigned to order 5?`
- `What is the total order value for all electronics orders?`

Watch the **log panel** on the right side of the chat window. You will see the agent deciding which tool to call, making the request, and then using the returned data to form its answer. This is the difference between a **LLM** (which can only generate text) and an **agent** (which can take actions and use real data).

Try asking something that requires **both tools**, like `What orders belong to customers in Europe?` The agent will need to check the customer table for European customers and then cross-reference that with the order data. It may take a couple of steps, and you can watch the reasoning in the logs.

---

10. When Things Go Wrong

AI agents do not always get it right. You may notice the agent occasionally calling the wrong tool, misinterpreting a result, or giving a slightly different answer to the same question (remember: **agents are not deterministic**). This is normal.

A few things that help:

- **Write clear tool descriptions.** The agent relies on these to pick the right tool.
- **Be specific in the system prompt** about what the agent should and should not do.
- **Use the log panel to debug.** If the agent gave a wrong answer, the logs will show you exactly which tool it called and what data came back.

---

11. Saving the Workflow & Next Steps

Save your workflow. You now have a working customer service agent that can remember conversations, look up customer records, and fetch order data. This is a basic example, but it covers the building blocks: **a trigger, an agent, a model, memory, and tools**.

From here you can add more tools, refine the prompt, or connect the agent to a live chat interface.

> **Learn More**
>
> - Advanced AI documentation
> - AI workflow templates

📝 Quiz & Exam Bank — QA101

> **How to Read**
>
> Each question includes the prompt, the answer choices, and the answer key in a green callout with a short explanation.

Getting Started Quiz

Question 1 — App without an integration node

Question: What can you do if there is no n8n node for an app/service you want to use in a workflow?

1. Ask the n8n team to create the node
2. Create the node yourself
3. Use the HTTP Request node instead

> ✅ **Answer: **Use the HTTP Request node instead****
>
> HTTP Request is the versatile solution for calling any API that does not yet have a dedicated node. *(Note: this question previously scored 0.0 — make sure to answer with the HTTP Request option.)*

Question 2 — Trigger Nodes

Question: Which of the following are **Trigger Nodes**?

1. `Schedule node`
2. `Code node`
3. `Airtable node`
4. `Edit Fields node`

> ✅ **Answer: ** `Schedule node` ****
>
> Only the Schedule Trigger starts a workflow automatically. Code, Airtable, and Edit Fields are regular processing nodes.

Question 3 — How the Code node executes code

Question: How does the Code node execute code?

1. The code gets executed once for all items
2. The code can only be executed by a Schedule Trigger
3. The code runs continuously in the background until manually stopped
4. The code gets executed once for every item

> ✅ **Answer: **The code gets executed once for all items****

> The default Code node mode is *Run Once for All Items*. A *Run Once for Each Item* mode also exists, but it is not the default behavior.

Question 4 — IF vs Switch

Question: What is the difference between the IF node and the Switch node?

1. None, they are the same
2. The IF node supports two conditional routes, the Switch node supports unlimited conditional routes
3. The IF node supports unlimited conditional routes, the Switch node supports two conditional routes

> ✅ **Answer: **IF supports two routes, Switch supports many routes****
>
> IF = a `True` / `False` branch. Switch = multiple branches based on the rules you define.

Question 5 — Correct data structure

Question: What is the correct data structure?

1. `{[json: {jsonKeyName: 'keyValue'}]}`
2. `[{json: {jsonKeyName: 'keyValue'}}]`
3. `json: [jsonKeyName: 'keyValue']`

> ✅ **Answer: ** `[{json: {jsonKeyName: 'keyValue'}}]` **
>
> n8n uses an **array of objects**, and each object is wrapped in a `json` key.

Question 6 — Publishing a workflow with a Schedule Trigger

Question: What happens when you publish a workflow that has a Schedule Trigger node?

1. The workflow runs once immediately and then stops until you manually trigger it again
2. The workflow is shared publicly so other n8n users on your instance can edit it
3. The workflow will run automatically at the scheduled time without any manual action
4. The Schedule Trigger still needs to be manually enabled separately from publishing

> ✅ **Answer: **The workflow runs automatically on schedule****
>
> After publishing, the Schedule Trigger is active in the background and runs the workflow according to its schedule.

Question 7 — Causes of node errors

Question: What can be the reasons for node errors?

1. The node credentials are missing or incorrect
2. The node parameters are not configured correctly
3. You don't have permission to use that node

> ✅ **Answer: **All of the above****
>
> Errors can come from credentials, parameter configuration, or access permissions.

Question 8 — n8n data structure

Question: What is the data structure of n8n?

1. A string of JSON
2. An array of objects
3. A single object
4. An array of arrays

> ✅ **Answer: **An array of objects****
>
> Data between nodes is always an array, where each element is an object `{ json: ... }`.

Question 9 — Node action per data unit

Question: An n8n node performs its action on each _____ of incoming data.

1. `key`
2. `array`
3. `workflow`
4. `item`

> ✅ **Answer: ** `item` ****
>
> n8n nodes generally process data per **item** automatically.

Working with Data Quiz

Question 10 — Referencing a node that is not directly connected

Question: You need to reference a field called `region` from a node named `GetCountries` that is **not directly connected** to the current node. Which expression is correct?

1. `{{ $json.region }}`
2. `{{ $('GetCountries').item.json.region }}`
3. `{{ $input('GetCountries').json.region }}`
4. `{{ $json.GetCountries.region }}`

> ✅ **Answer: ** `{{ $('GetCountries').item.json.region }}` ****
>
> Use `$('NodeName')` to pull data from another node, then `.item` for the item at the same index.

Question 11 — Adding 20 years to a date

Question: In an Edit Fields node, which expression adds 20 years to a date field called `customerSince`?

1. `{{ $json.customerSince + years(20) }}`
2. `{{ $json.customerSince.toDateTime().plus(20, 'years') }}`
3. `{{ DateTime.fromISO($json.customerSince) + 20 }}`
4. `{{ $json.customerSince.addYears(20) }}`

> ✅ **Answer: ** `{{ $json.customerSince.toDateTime().plus(20, 'years') }}` ****
>
> n8n uses Luxon. Convert first with `.toDateTime()`, then use `.plus(20, 'years')`.

Question 12 — Continue Using Error Output

Question: What happens when you set a node's On Error setting to **Continue Using Error Output**?

1. Failed items are silently skipped and only successful items continue to the next node
2. The workflow stops execution and the error is written to the log
3. A second output appears on the node and failed items are routed there
4. The node retries the operation up to three times before failing

> ✅ **Answer: **A second output appears and failed items are routed there****
>
> The node gets an additional (red) *error output* dedicated to failed items.

Building an AI Agent Quiz

Question 13 — Standard workflow vs AI agent

Question: What is a key difference between a standard n8n workflow and an AI agent workflow?

1. AI agents do not need a trigger node
2. Standard workflows are deterministic, AI agents are not
3. Standard workflows cannot use HTTP Request nodes
4. AI agent workflows run faster

> ✅ **Answer: **Standard workflows are deterministic, AI agents are not****
>
> Standard workflows follow a fixed path. AI agents make decisions (such as choosing tools), so their results can vary.

Question 14 — How the AI Agent chooses a tool

Question: How does the AI Agent decide which tool to call?

1. You must configure a condition to tell it which tool to use
2. It reads the tool description and matches it to the user's question
3. Tools are called in the order they were added to the canvas

4. It calls every tool on each message and picks the best result

> ✅ **Answer: **It reads the tool description and matches it to the user's question****
>
> That is why tool descriptions must be clear and specific.

Question 15 — Purpose of the system message

Question: What does the system message (system prompt) control in the AI Agent?

1. The memory window size
2. How many tools the agent can call per message
3. The agent's behavior, personality, and instructions
4. Which language model the agent uses

> ✅ **Answer: **The agent's behavior, personality, and instructions****
>
> The system prompt defines the agent's role, answer style, and boundaries.

Question 16 — Why Simple Memory is not for production

Question: Why is Simple Memory not recommended for production use?

1. It does not work with OpenAI models
2. It can only remember one message at a time
3. It costs more than database-backed memory options
4. It only stores messages in memory, so they are lost on restart

> ✅ **Answer: **It only stores messages in memory, so they are lost on restart****
>
> For production, use **Postgres Chat Memory** or **Redis Chat Memory** so conversations persist.

✅ Wrap-up

🎉 Course Complete

Congratulations on completing the n8n Quickstart. You started with an empty *canvas* and built your way up to a working AI agent that can look up real data and answer questions. Along the way you learned how to wire up triggers and HTTP requests, transform and merge data across nodes, handle errors before they become problems, and connect an AI model to external tools.

🚀 Where to Go from Here

The n8n Quickstart is designed to get you productive fast. If you want to go deeper, the **n8n Foundations** program covers the full platform across three courses:

Course	Focus
Essentials: Your First Workflows (N8N101)	The canvas, core nodes, data flow, and building your first integrations from scratch.
Integrations: APIs and Connected Workflows (N8N102)	HTTP requests, webhooks, API authentication, flow logic with IF/Switch/Loop nodes, sub-workflows, Data Tables, and error handling. All exercises use practice APIs hosted by the Academy, so no external accounts are needed.
In Practice: AI, Testing and Best Practices (N8N103)	AI-powered workflows using the AI Agent node, testing and debugging with the execution log and Try/Catch, and workflow organization with naming conventions, version control, and production readiness.

Complete all three Foundations courses to earn the **n8n Foundations program badge**. An official n8n certification exam is also on the way, giving you a way to validate your skills across the full program.

💬 Feedback & Community

This course is designed to give you the best possible learning experience. If you have suggestions for how the team can improve — whether that's content, pacing, exercises, or anything else — let them know.

- Give feedback
- **Online community:** join discussions, ask questions, and share workflows on the official n8n forum: community.n8n.io
- **Local meetups and events:** community members and ambassadors host meetups where you can meet other n8n users, attend talks or workshops, and network. Upcoming events are announced through community channels and listed on the Luma page.

📌 Appendix — Node Cheat Sheet

Node	Main Purpose	Important Notes
Manual Trigger	Run a workflow manually during testing.	Keep it even when an automatic trigger exists.
Schedule Trigger	Run a workflow on a schedule.	Requires the workflow to be **Published**; follows the Workflow Settings timezone.
HTTP Request	Call any API.	The fallback when no dedicated node exists; configure method, headers, auth.
Data Table	Internal database similar to a spreadsheet.	Workflow & table must be in the same workspace; use Upsert to avoid duplicates.
If	Two-way branching (`True` / `False`).	Use for if-then-else logic.
Switch	Multi-way branching.	Use for more than 2 conditions.
Edit Fields (Set)	Select/add/overwrite fields.	Reduce payload before entering a database.
Code	Run custom JavaScript/Python.	Choose *Run Once for All Items* or *Run Once for Each Item*.
Discord	Send messages to Discord.	Can use a Webhook without a bot.
Split Out	Split an array into separate items.	Set *Field To Split Out* to the array name.
Merge	Combine data from multiple sources.	Append/Combine/Choose Branch modes; use dot-notation for nested keys.
Loop Over Items	Process data in batches.	Set `Batch Size = 1` for per-item handling; stops automatically.
HTML	Web scraping with CSS Selectors.	Enable *Return Array* for multiple elements.
XML	Convert XML ↔ JSON.	Useful for legacy systems.
Date & Time	Change/calculate/round dates.	Enable *Include Input Fields* to keep old columns.
Edit Image	Rotate, crop, resize, composite.	Works on binary data.
Error Trigger	Start an *error workflow*.	Only active in production executions; cannot be tested manually.
AI Agent	Run an agent with a model, memory, and tools.	Needs a clear system prompt.
Simple Memory	Store a window of conversation.	Default 5 interactions; not persistent — use Postgres/Redis for production.
Data Table Tool	Give the agent access to a Data Table.	Set operation to `Get` ; use a descriptive name like `GetCustomers` .